

8. sklearn及其基本使用

sklearn是用于机器学习的工具包。机器学习本质上就是根据数据训练一个数学模型，并对未知数据能够成功进行合理预测。换言之，机器学习的要素包括训练材料与算法（数据、限制条件）、训练目的（分类、回归或其他）、训练效果的衡量标准（目标函数）。事实上，机器学习也就是人工智能的现代基础。

机器学习的任务大体上按照有无标签区分。有标签指导训练过程的任务被称作有监督学习，包含目标为离散标签的分类问题和目标为连续标签的回归问题。无标签指导的任务被称作无监督学习，主要包括聚类问题和降维问题等。此外，还有半监督和强化学习等问题，这些问题我们不作过多讨论。

- 分类问题：比如机器学习中著名的鸢尾花数据集（鸢尾花也就是水仙花），数据集收集了150朵花的萼片长度、萼片宽度、花瓣长度和花瓣宽度等四个形状指标，最后有一列标注数据表明每一条数据数据是哪种鸢尾花（数据集的最后一列是字符串，有setosa, versicolor和virginica三种类型）。数据的最后一列显然是离散变量，而前面四列自变量都是连续变量。这种数据可以用来预测一朵新的鸢尾花是三种鸢尾花中的哪一种，但是分类模型需要基于后续的标签来指导，这叫分类问题。也可以把它抽象成：已知自变量的数值，求因变量数据属于ABC三类中哪一类的选择题。分类问题本身就像做选择题一样，有标准答案。
- 回归问题：也就是在已知自变量的情况下若因变量是连续数据如何去进行建模与预测。例如，著名的波士顿房价数据集，在这个数据集中存在不同的自变量，但因变量房价却是连续的数值。对房价做预测本质上也是一种回归。也可以把它抽象成：已知自变量的数值，求因变量的计算结果，使其与实际结果偏差不大。回归问题本身就像做计算题一样，也有标准答案。
- 聚类问题：另一类比较有意思的就是聚类问题，比如在一家服装专卖店内老板会收集每个VIP用户的信息，例如年龄、性别、消费次数、卡内充值、消费偏好等，但这些都是自变量没有因变量。现在老板需要根据这一系列自变量对用户进行画像，将其分为若干个群，至于分多少群、每个群体有什么样的特征是由数据自变量所决定的。只有自变量没有因变量将数据分群的过程就是聚类，它也可以被抽象为根据自变量的数值做一个论述题，是没有标准答案的。
- 降维问题：主要探究如何用更少的变量表示原始数据，并且尽可能保留更多的信息。常见的降维方法包括主成分分析、因子分析、独立成分分析、t-SNE等。

针对这些任务，我们将sklearn当中的模型分解为三个小板块：

- [8.1 数据集的预处理](#)
- [8.2 有监督学习的案例](#)
- [8.3 无监督学习的案例](#)

8.1 数据集的预处理

8.1.1 特征的规约与编码

在机器学习中，对数据集的特征进行编码和缩放是预处理步骤中非常重要的一部分。这些步骤有助于提升模型的性能，特别是当数据集中的特征具有不同的量纲或包含分类变量时。以下是几种常见的方法，包括使用 `sklearn` 库来实现它们。

1. 特征编码

独热编码（One-Hot Encoding）

独热编码通常用于将分类数据（如性别、国家等）转换为机器学习算法能够处理的数值形式。

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

# 示例数据
data = np.array([[ 'male' ], [ 'female' ], [ 'male' ], [ 'female' ]]).reshape(-1, 1)

# 初始化编码器
encoder = OneHotEncoder(sparse=False)

# 拟合并转换数据
encoded_data = encoder.fit_transform(data)

print(encoded_data)
```

标签编码（Label Encoding）

标签编码将每个类别映射到一个从0开始的整数。注意，它假设类别之间存在某种顺序，这在很多情况下并不成立。

```
from sklearn.preprocessing import LabelEncoder

# 示例数据
labels = ['male', 'female', 'male', 'female']

# 初始化编码器
encoder = LabelEncoder()

# 拟合并转换数据
encoded_labels = encoder.fit_transform(labels)

print(encoded_labels)
```

2. 特征缩放

标准化 (Standardization)

标准化是将特征缩放到均值为0，标准差为1的过程。这对于很多基于距离的算法（如K-近邻、K-均值聚类等）特别有用。

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# 示例数据
data = np.array([[1, 2], [3, 4], [5, 6]])

# 初始化标准化器
scaler = StandardScaler()

# 拟合并转换数据
scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

归一化 (Normalization)

归一化是将特征缩放到一个小的特定区间，通常是[0, 1]。这在很多算法中也是有用的，特别是当不同特征的取值范围差异很大时。

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 示例数据
data = np.array([[1, 2], [3, 4], [5, 6]])

# 初始化归一化器
scaler = MinMaxScaler()

# 拟合并转换数据
scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

总结

以上是使用 `sklearn` 库对数据集中的特征进行编码和缩放的一些常见方法。根据你的具体需求和数据集的特性，你可以选择最适合你的方法。注意，对于不同的算法和数据集，预处理步骤的选择可能会有所不同。

8.1.2 数据集切分与交叉验证

在 `sklearn`（`scikit-learn`）中，对数据集进行划分以及进行交叉验证是模型评估过程中非常常见的步骤。这些数据集划分方法帮助我们更好地了解模型在未见数据上的性能。

数据集划分

对于数据集的划分，我们通常使用 `train_test_split` 函数从 `sklearn.model_selection` 模块。这个函数允许我们将数据集分割成训练集和测试集，并且可以指定测试集的比例或大小。

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X = iris.data
y = iris.target

# 划分数据集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 现在X_train, y_train是训练集, X_test, y_test是测试集
```

在上面的代码中, `test_size=0.3` 表示测试集占整个数据集的30%, `random_state` 是一个随机种子, 用于确保每次划分的结果都是一样的。

交叉验证

交叉验证是一种评估统计分析方法, 它重复地划分数据为训练集和测试集, 每次使用不同的划分, 并计算平均性能指标。在 `sklearn` 中, 有几种交叉验证的策略。

1. K折交叉验证 (K-Fold Cross-Validation)

使用 `KFold` 或 `cross_val_score` 来进行K折交叉验证。这里以 `cross_val_score` 为例, 因为它直接返回了每次迭代的评分。

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# 假设你已经有了X_train, y_train
model = LogisticRegression()

# 执行5折交叉验证
scores = cross_val_score(model, X_train, y_train, cv=5)

print(scores) # 每一折的评分
print(scores.mean()) # 所有折的平均评分
```

2. 留一交叉验证 (Leave-One-Out Cross-Validation, LOOCV)

当数据集很小时, 可以使用留一交叉验证, 它每次留一个样本作为测试集, 其余作为训练集。

```
from sklearn.model_selection import LeaveOneOut
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
loo = LeaveOneOut()

# 这里需要自己编写循环来拟合和评分
scores = []
for train_index, test_index in loo.split(X_train):
    X_train_cv, X_test_cv = X_train[train_index], X_train[test_index]
    y_train_cv, y_test_cv = y_train[train_index], y_train[test_index]
    model.fit(X_train_cv, y_train_cv)
    score = model.score(X_test_cv, y_test_cv)
    scores.append(score)

print(scores)
print(np.mean(scores))
```

注意： `cross_val_score` 函数也可以直接使用 `cv=LeaveOneOut()` 来实现留一交叉验证，但上面的例子展示了如何手动进行这一过程，以便更好地理解其背后的逻辑。

总结

`sklearn` 提供了灵活的工具来划分数据集和进行交叉验证，这对于评估机器学习模型的性能至关重要。通过 `train_test_split` 可以轻松地将数据集划分为训练集和测试集，而 `cross_val_score` 和其他交叉验证工具则可以帮助我们更全面地评估模型在不同数据子集上的表现。

8.1.3 特征选择方法

在 `sklearn`（`scikit-learn`）中，特征选择与特征降维是机器学习过程中的重要步骤，它们可以帮助减少模型的复杂度，提高模型的泛化能力，并提升模型的性能。以下是关于如何在 `sklearn` 中进行特征选择与特征降维的详细解释：

特征选择

特征选择是从原始特征集中选择出对模型预测最有用的特征子集的过程。`sklearn` 提供了多种特征选择的方法，包括过滤方法、包装方法和嵌入方法。

过滤方法 (Filter Methods)

过滤方法使用统计学或信息论方法来评估每个特征的相关性，并选择最相关的特征。常用的过滤方法包括：

1. 方差选择法 (Variance Threshold) :

- 移除所有方差低于某个阈值的特征。这种方法假设方差小的特征可能是不重要的或冗余的。
- 在 `sklearn` 中，可以使用 `VarianceThreshold` 类来实现。

2. 互信息法 (Mutual Information) :

- 评估每个特征和目标变量之间的互信息，并选择具有高互信息的特征。
- 在 `sklearn` 中，可以使用 `mutual_info_classif` (分类问题) 和 `mutual_info_regression` (回归问题) 函数来计算互信息，并使用 `SelectKBest` 类来选择特征。

包装方法 (Wrapper Methods)

包装方法使用模型的性能作为特征选择的指标，并尝试找到最优的特征子集。常用的包装方法包括：

• 递归特征消除 (Recursive Feature Elimination, RFE) :

- 从初始的特征集开始，使用模型对特征进行排序，并删除最不重要的特征，直到达到所需的特征数量。
- 在 `sklearn` 中，可以使用 `RFE` 或 `RFECV` (带有交叉验证的RFE) 类来实现。

嵌入方法 (Embedded Methods)

嵌入方法将特征选择作为模型训练过程的一部分，并在学习过程中选择最优的特征子集。常用的嵌入方法包括：

• 基于L1正则化的方法:

- 使用L1范数作为正则化项，对模型参数进行惩罚，从而降低模型复杂度并选择有用的特征。
- 在 `sklearn` 中，可以使用 `Lasso` 回归模型来实现L1正则化，并通过选择具有非零系数的特征来进行特征选择。

• 基于树的方法:

- 决策树和随机森林等基于树的方法可以在训练过程中自动评估特征的重要性，并可以用于特征选择。
- 在 `sklearn` 中，可以使用 `feature_importances_` 属性来获取特征的重要性，或使用 `SelectFromModel` 类结合基于树的模型来进行特征选择。

当然可以。以下是针对几种常见的特征选择方法的代码用法和代码案例，这些示例使用了 `sklearn` 库。

1. 方差选择法 (Variance Threshold)

```
from sklearn.feature_selection import VarianceThreshold
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 应用方差选择法，移除方差小于0.8的特征
selector = VarianceThreshold(threshold=0.8)
X_transformed = selector.fit_transform(X)

# 查看变换后的特征数量
print(X_transformed.shape)
```

注意：这里的 threshold=0.8 是示例值，实际应用中应根据数据特性调整。

2. 互信息法 (使用SelectKBest)

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用互信息法选择最好的两个特征
selector = SelectKBest(mutual_info_classif, k=2)
X_new = selector.fit_transform(X, y)

# 查看选择的特征索引
print(selector.get_support(indices=True))

# 查看变换后的特征
print(X_new.shape)
```


3. 递归特征消除 (RFE)

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用逻辑回归作为基模型，递归特征消除选择5个特征
estimator = LogisticRegression()
selector = RFE(estimator, 5, step=1)
X_new = selector.fit_transform(X, y)

# 查看选择的特征索引
print(selector.support_)
# 查看变换后的特征
print(X_new.shape)
```

4. 基于L1正则化的特征选择 (使用Lasso)

```
from sklearn.linear_model import Lasso
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用Lasso回归并选择alpha参数
lasso = Lasso(alpha=0.1)
lasso.fit(X, y)

# 查看哪些特征的系数非零，即选择了哪些特征
print(lasso.coef_ != 0)

# 注意：这里通常不会直接转换X，而是基于coef_的结果来选择特征
# 例如，可以创建一个新的DataFrame或数组，只包含coef_非零对应的特征
```

5. 嵌入方法（以随机森林为例）

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用随机森林作为基模型
clf = RandomForestClassifier(n_estimators=10)
clf.fit(X, y)

# 使用SelectFromModel选择重要性高于平均的特征
selector = SelectFromModel(clf, prefit=True, threshold='mean')
X_new = selector.transform(X)

# 查看选择的特征数量
print(X_new.shape)

# 查看哪些特征被选择了
print(selector.get_support())
```

注意：以上代码中的 `threshold` 参数在 `SelectFromModel` 中根据基模型的不同可能有所不同，这里使用 `'mean'` 作为示例，意味着选择重要性高于平均的特征。在实际应用中，你可能需要根据模型的 `feature_importances_` 来调整这个阈值。

特征降维

特征降维是将高维特征空间转换为低维特征空间的过程，以减少数据的复杂性和计算成本。sklearn 提供了多种特征降维的方法，包括主成分分析（PCA）和线性判别分析（LDA）等。

主成分分析（PCA）

PCA是一种常用的线性降维方法，它通过保留数据中的主要特征（即方差最大的方向）来降低数据的维度。在 sklearn 中，可以使用 `PCA` 类来实现PCA降维。

- 优点：
 - 能够有效降低数据的维度，同时保留数据中的大部分信息。
 - 可以去除数据中的噪声和冗余信息。

- **缺点：**

- 是一种线性降维方法，对于非线性关系的数据可能效果不佳。
- 降维后的特征可能难以解释。

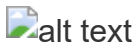
线性判别分析 (LDA)

LDA是一种有监督的降维方法，它试图找到一种线性组合，使得类内方差最小化而类间方差最大化。然而，需要注意的是，在 `sklearn` 中，LDA通常用于分类任务（通过 `LinearDiscriminantAnalysis` 类），并且它本身并不直接用于降维（尽管在分类过程中会隐含地进行特征变换）。对于降维目的，更常使用的是PCA或其他无监督的降维方法。

8.2 有监督学习的案例

8.2.1 分类问题：鸢尾花

鸢尾花数据集是机器学习中非常常用的一个数据集，在`sklearn.dataset`中自带，通过`load_iris()`函数导入。鸢尾花数据集是一个用于分类问题的经典数据集，包含三种不同种类的鸢尾花：山鸢尾、变色鸢尾和维吉尼亚鸢尾。数据集中的每个样本都有四个特征：花萼长度、花萼宽度、花瓣长度和花瓣宽度。这四个特征是用来预测鸢尾花卉属于哪一品种的重要依据。每一类各50条样本。我们可以看到它的统计分布图：



当我们在对数据集进行统计描述时，我们通常会关注数据的中心趋势、分散程度、分布形状和相关性等特征。我们可以对Iris数据集进行如下统计描述：

- **中心趋势：**中心趋势描述了数据集中的观测值倾向于聚集的数值。对于Iris数据集中的量化特征（如花萼片长度、萼片宽度、花瓣长度和花瓣宽度）都有一定的中心趋势，我们可以通过计算平均值来了解每个特征的中心位置。而对于分类特征（如花种），我们可以通过统计每个类别的出现频率来描述其中心趋势。在Iris数据集中，*Setosa*、*Versicolour*和*Virginica*三种花种各有50个样本，各占总样本数的1/3。
- **分散程度：**Iris数据集中的每个特征都有一定的分散程度，可以通过计算其标准差来描述。分散程度反映了数据点之间的差异程度，标准差越大，数据点之间的差异就越大。例如，*Setosa*花种的花瓣长度的标准差为0.82厘米，*Versicolour*花种的花瓣长度的标准差为1.76厘米，*Virginica*花种的花瓣长度的标准差为2.06厘米。*Setosa*花种的花瓣长度标准差较小，表明大多数样本的花瓣长度接近平均值。相反，如果一个特征的标准差较大，如*Virginica*花种的花瓣长度，这可能表明样本之间存在较大的变异。

- 分布形状：分布形状描述了数据的分布模式，是对称、偏斜还是具有峰度。Iris数据集中的每个特征都有一定的分布形状，我们可以通过绘制直方图或密度图来观察数据的分布形状。分布形状反映了数据点的分布特征，对称的分布形状（如正态分布）通常表示数据的中心趋势与分散程度相符，非对称的分布形状则可能表示数据集中于某一范围或两侧。例如，如果Setosa花种的花瓣长度呈现右偏态分布，这可能意味着大多数样本的花瓣长度较短，只有少数样本的花瓣长度较长。这种分布形状可以帮助我们了解数据的集中趋势和异常值的可能性。
- 相关性：相关性描述了数据集中不同特征之间的关系，反映了特征之间的相关程度，相关性越高，特征之间的变化趋势越相似。通过计算相关系数，我们可以量化两个特征之间的线性关系强度。例如，如果花瓣长度和花瓣宽度之间的相关系数接近1，这表明这两个特征之间存在强烈的正相关关系，即花瓣长度较长的花往往花瓣宽度也较大。

进行机器学习第一步是导入数据并处理数据。通过下面的代码导入数据：

```

import numpy as np

import pandas as pd

# 鸢尾花数据集，红酒数据集，乳腺癌数据集，房价预测数据集

from sklearn.datasets import load_iris, load_wine, load_breast_cancer, load_boston

# 回归重要指标

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# 分类重要指标

from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_recall_curve,

from sklearn.model_selection import train_test_split #训练集训练集分类器

import graphviz #画文字版决策树的模块

import pydotplus #画图片版决策树的模块

from IPython.display import Image #画图片版决策树的模块

iris = load_iris()

print(iris.data) # 数据

print(iris.target_names) # 标签名

print(iris.target) # 标签值

print(iris.feature_names) # 特征名(列名)

iris_dataframe = pd.concat([pd.DataFrame(iris.data), pd.DataFrame(iris.target)], axis=1)

print(iris_dataframe)

Xtrain, Xtest, Ytrain, Ytest = train_test_split(iris.data, iris.target, test_size=0.3)

```

随后选择对应接口创建模型，输入数据通过fit方法进行训练，然后进行predict并评估指标即可。代码如下：

```
from sklearn.linear_model import LogisticRegression,LinearRegression

from sklearn.neighbors import KNeighborsRegressor,KNeighborsClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.tree import DecisionTreeRegressor,DecisionTreeClassifier

from sklearn.ensemble import RandomForestRegressor,RandomForestClassifier

from sklearn.ensemble import ExtraTreesRegressor,ExtraTreesClassifier

from sklearn.ensemble import AdaBoostRegressor,AdaBoostClassifier

from sklearn.ensemble import GradientBoostingRegressor,GradientBoostingClassifier

clf = RandomForestClassifier()

clf.fit(Xtrain, Ytrain)

Ypredict=clf.predict(Xtest)

print(r2_score(Ypredict,Ytest))
```

其中，决策树、随机森林等具有树形结构的基学习器可以把树形结构打印出来并保存为PDF或png文件，代码如下：

```

from sklearn import tree

tree_data = tree.export_graphviz(

    clf

    ,feature_names =iris.feature_names

    ,class_names = iris.feature_names#也可以自己起名

    ,filled = True #填充颜色

    ,rounded = True #决策树边框圆形/方形

)

graph1 = graphviz.Source(tree_data.replace('helvetica','Microsoft YaHei UI'), encoding='utf-8')

graph1.render('./iris_tree')

```

Python中想要评估模型效果的话scikit-learn也提供了不同问题的评估指标接口。

Scikit-learn（简称sklearn）是一个非常强大的机器学习库，提供了很多用于分类、回归和聚类的评估指标函数。以下是常用的一些函数：

分类问题常用函数：

- `accuracy_score`: 输入参数包括`y_true`（真实标签）和`y_pred`（预测标签）。输出参数为准确率，即预测正确的样本数占总样本数的比例。它的作用是评估分类器的准确率，通过比较真实标签和预测标签的一致性来计算准确率。
- `confusion_matrix`: 输入参数包括`y_true`（真实标签）和`y_pred`（预测标签）。输出参数为混淆矩阵，展示各类别之间的预测和实际分类情况。它的作用是评估分类器的性能，通过比较真实标签和预测标签的分类情况来生成混淆矩阵，以量化分类器的正确率、精度、召回率和F1分数等指标。
- `classification_report`: 输入参数包括`y_true`（真实标签）和`y_pred`（预测标签）。输出参数为分类报告，包括精确度、召回率和F1分数等分类性能指标。它的作用是提供详细的分类性能评估报告，通过比较真实标签和预测标签的分类情况来计算各类别的精确度、召回率和F1分数，以全面评估分类器的性能。
- `roc_auc_score`: 输入参数包括`y_true`（真实标签）和`y_pred`（预测概率）。输出参数为ROC曲线下的面积，用于评估二元分类器的性能。它的作用是通过计算ROC曲线下的面积来评估分类器的性能，ROC曲线展示了不同分类阈值下真正例率（TPR）和假正例率（FPR）的变化情况，AUC值越大表示分类器性能越好。

- `average_precision_score`: 输入参数包括`y_true`（真实标签）和`y_score`（预测分数）。输出参数为平均精度，用于评估二元分类器的性能。它的作用是计算在不同分类阈值下的平均精度，综合考虑了真正例率（TPR）和假正例率（FPR），以更全面地评估分类器的性能。
- `brier_score_loss`: 输入参数包括`y_true`（真实标签）和`y_prob`（预测概率）。输出参数为Brier分数，用于评估二元或多元分类器的性能。它的作用是计算Brier分数，通过比较真实标签和预测概率的差异来评估分类器的性能。Brier分数越小表示预测概率与真实标签越接近，分类器性能越好。
- `f1_score`: 输入参数包括`y_true`（真实标签）和`y_pred`（预测标签）。输出参数为F1分数，用于评估分类器的性能。它的作用是计算F1分数，综合考虑了精确度和召回率，以更全面地评估分类器的性能。F1分数越高表示分类器性能越好。

8.2.2 回归问题：波士顿房价

波士顿房价数据集是一个非常著名的数据集，广泛用于回归分析和机器学习的入门研究。该数据集最初由哈里森和鲁宾菲尔德在1978年发布，包含了波士顿地区房价的中位数与各种影响房价的因素。具体来说，这个数据集包含506个数据点，每个数据点有14个属性，这些属性包括城镇人均犯罪率、住宅用地比例、城镇非零售商用土地比例、查尔斯河虚拟变量、一氧化氮浓度、住宅平均房间数、1940年之前建成的自用房屋比例、到五个波士顿就业中心的加权距离、辐射性公路的接近指数、每10000美元的全值财产税率、城镇师生比例、城镇中黑人的比例、人口中地位较低人群的百分比以及自有住房的中位数价值（单位：千美元）。

房价预测的任务是基于这些数据集中的特征（即自变量），使用统计分析、机器学习或深度学习等方法，来构建一个预测模型，以预测波士顿地区房价的中位数（即因变量）。这是一个典型的回归任务，因为预测目标是一个连续的实数值。通过训练模型，我们可以发现哪些因素对房价有显著影响，并据此对未来房价进行预测。

导入数据、数据预处理的方法与前面一样，这里我不多赘述。那么可以使用的回归模型也列举在了上面，甚至训练方法都一模一样。


```

from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error
import numpy as np

# 加载波士顿房价数据集
boston = load_boston()
X = boston.data
y = boston.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 线性回归
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print("线性回归的MSE:", mean_squared_error(y_test, y_pred_lr))

```

回归问题常用评估函数：

mean_squared_error: 输入参数包括y_true（真实值）和y_pred（预测值）。输出参数为均方误差，即预测值与真实值差的平方的平均值。它的作用是衡量回归模型的预测误差，通过比较预测值和真实值之间的差异来评估模型的性能。

- **mean_absolute_error:** 输入参数包括y_true（真实值）和y_pred（预测值）。输出参数为平均绝对误差，即预测值与真实值差的绝对值的平均值。它的作用是衡量回归模型的预测误差，通过比较预测值和真实值之间的差异来评估模型的性能。
- **median_absolute_error:** 输入参数包括y_true（真实值）和y_pred（预测值）。输出参数为中位数绝对误差，即预测值与真实值差的中位数绝对值。它的作用是衡量回归模型的预测误差，通过比较预测值和真实值之间的差异来评估模型的性能。
- **r2_score:** 输入参数包括y_true（真实值）和y_pred（预测值）。输出参数为R平方值，衡量回归模型的拟合优度。它的作用是通过计算R平方值来评估模型对数据的拟合程度，R平方值越接近于1表示模型拟合越好。
- **explained_variance_score:** 输入参数包括y_true（真实值）和y_pred（预测值）。输出参数为解释方差，衡量模型对数据的解释程度。它的作用是通过计算解释方差来评估模型对数据的解释能力，解释方差越接近于1表示模型对数据的解释程度越高。

8.3 无监督学习的案例

8.3.1 聚类问题

以下是使用 `sklearn` 库进行KMeans聚类和DBSCAN聚类的代码demo。

KMeans聚类

KMeans是一种基于距离的聚类算法，它将数据划分为预定义数量的簇（K），并使得簇内方差最小化。

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

# 生成模拟数据
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# 应用KMeans聚类
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# 可视化结果
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75);
plt.title("KMeans Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

DBSCAN聚类

DBSCAN是一种基于密度的聚类算法，它可以将具有足够高密度的区域划分为簇，并能在具有噪声的数据集中发现任意形状的簇。

```

from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# 生成模拟数据
X, _ = make_moons(n_samples=300, noise=0.1, random_state=42)

# 应用DBSCAN聚类
db = DBSCAN(eps=0.2, min_samples=5).fit(X)
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
labels = db.labels_

# 可视化结果
unique_labels = set(labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black used for noise.
        col = [0, 0, 0, 1]

    class_member_mask = (labels == k)

    xy = X[class_member_mask & core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=14)

    xy = X[class_member_mask & ~core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=6)

plt.title("DBSCAN Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```

注意：在DBSCAN的demo中，我使用了 `numpy` 库来生成颜色数组，但我没有在代码中显式导入它（你应该在代码顶部添加 `import numpy as np`）。另外，由于DBSCAN可以识别噪声点（即不属于任何簇的点），因此我使用了 `-1` 标签来表示这些噪声点，并在可视化时将它们着色为黑色。

这些代码demo提供了KMeans和DBSCAN聚类的基本使用方法，并包括了数据生成、聚类模型的训练和结果的可视化。

8.3.2 降维问题

以下是使用 `sklearn` 库进行PCA（主成分分析）和LDA（线性判别分析）降维的代码案例。

PCA降维

PCA是一种无监督的降维技术，它通过保留数据中的最大方差方向来减少数据的维度。

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# 加载数据集
iris = load_iris()
X = iris.data
y = iris.target

# 应用PCA降维到2维
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# 可视化结果
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of IRIS dataset')
plt.colorbar()
plt.show()
```

LDA降维

LDA是一种有监督的降维技术，它旨在最大化类间方差并最小化类内方差，通常用于分类任务。

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# 加载数据集
iris = load_iris()
X = iris.data
y = iris.target

# 应用LDA降维到2维
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X, y)

# 可视化结果
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.title('LDA of IRIS dataset')
plt.colorbar()
plt.show()
```

在这两个例子中，我们首先加载了Iris数据集，这是一个常用于分类任务的多变量数据集。然后，我们分别应用了PCA和LDA算法，将数据的维度从4维降到了2维，以便进行可视化。在可视化部分，我们使用了 `matplotlib` 库来绘制散点图，其中不同的颜色代表不同的类别。

请注意，由于LDA是一种有监督学习方法，我们在调用 `fit_transform` 时需要同时提供特征矩阵 `X` 和标签 `y`。而PCA是无监督的，因此在调用 `fit_transform` 时只需要提供特征矩阵 `X`。

扩展阅读

我们有关机器学习的项目属实是太多了，多到看不过来。但是我这里力推两个大项目：

- 南瓜书：[pumpkin-book](#)
- 李宏毅机器学习：[leedl-tutorial](#)

这两都是被广泛转载阅读的经典，大家可以赶紧拖下来看看。